

Cognitive Court: A Multi-Model Framework for Scientific Reasoning Evaluation Using Large Language Models

Tejas Venu

pmm7nh

Email: pmm7nh@virginia.edu

Abstract—Large language models have recently demonstrated strong performance on scientific question-answering tasks. However, evaluating the quality of scientific reasoning generated by these models remains a difficult problem. Many systems evaluate only the final answer while ignoring whether the explanation contains coherent causal reasoning, sufficient mechanistic detail, or scientifically supported claims. This project presents *Cognitive Court*, a multi-model scientific reasoning evaluation framework designed to compare, verify, and judge answers generated by multiple language models. The system combines parallel answer generation, structured prompting, automated judging, claim verification, and interactive visualization into a unified architecture. Multiple language models independently answer the same scientific question, after which a judge model evaluates the responses using a detailed reasoning rubric. A separate verifier model analyzes factual support for claims made within each answer. The framework also includes a gatekeeper component that filters unsafe or irrelevant prompts before generation begins. In addition to a command-line evaluation pipeline, the project includes a Streamlit-based interface for interactive comparison of model outputs. This report describes the motivation behind the project, the system architecture, implementation details, the qualitative evaluation results, and the lessons learned from building the framework.

I. MOTIVATION

The motivation for this project came from a simple but important observation: a language model can produce an answer that looks convincing while still failing to explain the scientific mechanism behind it. In scientific question answering, a correct final statement is not always enough. A strong response should connect the conclusion to the underlying process, explain why the phenomenon occurs, and avoid unsupported claims that sound plausible but are not actually well grounded. This becomes especially important in educational settings, where the goal is not only to get the right answer but also to help the user understand the reasoning that leads to it.

Another reason for building this project was the difficulty of comparing scientific answers produced by different models. Different models often have different strengths, and those strengths are not always obvious from the final output alone. One model may produce a short and direct answer, another may provide a detailed explanation, and a third may produce a response that appears polished but lacks real depth. Without a structured comparison framework, it is hard to identify which answer is genuinely better. Human evaluation can help,

but it is slow, expensive, and difficult to repeat consistently across many questions. This project was built to make that comparison process more systematic and more transparent.

A further motivation was the desire to make the evaluation process itself more trustworthy. In many systems, the user only sees a final selected answer, but not why it was selected. That makes it difficult to understand whether the decision was based on correctness, reasoning quality, or simply formatting. Cognitive Court was designed to expose the full process, including the answers generated by each model, the judge’s scoring, the verifier’s analysis of claims, and the final selected winner. The result is not just an answering system, but a framework for studying reasoning quality in a way that is inspectable and easy to interpret.

The project was also motivated by the need for a safer and more controlled scientific question-answering environment. Not every user prompt should be passed directly to the models. Some questions are unsafe, irrelevant, or not suited for scientific evaluation. A gatekeeping stage was therefore included to filter out inappropriate prompts before the rest of the pipeline begins. This helped ensure that the system remains focused on scientific and educational use rather than arbitrary or harmful content. Overall, the project was built to support better scientific reasoning evaluation, more transparent model comparison, and more reliable interpretation of model behavior.

II. METHODS

The system was built as a modular Python application so that each part of the workflow could be developed, tested, and improved independently. This modular design was important because the project is not just about producing answers. It is about constructing a full evaluation pipeline in which different components work together: one part filters the question, another generates candidate responses, another verifies the claims inside those responses, and another judges which answer is strongest. Keeping these responsibilities separate made the system easier to understand and more flexible to extend.

The workflow begins with question filtering. Before a question is sent to the answer models, it is checked by a gatekeeper component that determines whether the prompt is appropriate for scientific answering. The purpose of this stage is to reject questions that are hateful, nonsensical, unsafe, overly personal,

or unrelated to the scientific task. If the question is acceptable, the system continues to the generation stage. If not, the system stops early and returns a neutral refusal. This early filtering step is important because it prevents the rest of the pipeline from processing prompts that are outside the intended scope of the project.

After the question passes the gatekeeper, the system generates answers using multiple language models in parallel. The framework supports three different answer-generation models, and each model independently produces a response to the same scientific question. The reason for using multiple models is that different models may provide different forms of reasoning. One may be concise, another may be more descriptive, and another may be stronger on mechanistic explanation. By comparing them side by side, the system can identify the answer that best balances correctness, completeness, and clarity.

The system supports both a standard answering mode and a chain-of-thought mode. In standard mode, the model is instructed to provide a direct answer without revealing intermediate reasoning. This mode tests how well the model can be concise while still remaining scientifically correct. In chain-of-thought mode, the model is guided to organize the response into a main answer, a detailed explanation, and explicit reasoning steps. This mode is useful for scientific questions because it encourages the model to make the causal chain visible rather than compressing everything into a short statement. The difference between these two modes also makes it easier to compare whether structured reasoning improves answer quality.

Prompt engineering plays a central role in the method because the behavior of both the answer models and the evaluation models depends strongly on how they are instructed. The answer prompts were written to encourage clarity and scientific relevance while avoiding unnecessary verbosity. The judge prompt was written to evaluate candidate answers using a rubric instead of a simple intuition-based choice. The rubric focuses on correctness, mechanism coverage, logical coherence, explanatory depth, and unsupported claims. These dimensions were chosen because they capture the main features of a strong scientific explanation. A good answer should be correct, but it should also show how the mechanism works, how the steps connect, and whether the answer adds meaningful scientific detail. The judge is explicitly instructed not to reward length alone. That design choice matters because a long response can still be shallow if it repeats the same idea in different words.

The verifier stage adds another layer of analysis. Rather than judging the quality of the explanation, the verifier checks whether the individual claims in an answer appear scientifically supported. It extracts claims from the answer and classifies them as supported, unsupported, or unclear. This is useful because a response can appear logically organized but still contain a weak or incorrect statement. The verifier therefore acts as a factual safety net that complements the judge’s reasoning-based evaluation. It also helps reduce the chance that a polished but unsupported response is selected as

the final winner.

The system uses a shared client abstraction to interact with the different models. This design allows the project to treat model interaction in a uniform way, regardless of whether the model is being used for answer generation, judging, or verification. The abstraction makes it easier to switch between different providers and helps keep the rest of the code simple. One practical issue that arose during development was the fact that language models sometimes return malformed JSON, even when explicitly asked to return valid structured output. Because the judge and verifier must produce machine-readable results, the system includes repair logic that attempts to recover valid JSON from imperfect responses. This made the pipeline much more robust and reduced the number of failures caused by formatting issues.

The evaluation pipeline then coordinates the entire process. The question is loaded, the candidate answers are generated, the verifier analyzes each answer, and the judge selects the winner. Before judging, the answers are anonymized using neutral labels so that the model identity does not affect the final decision. This step is important because it reduces bias and forces the judge to focus only on the content of the responses. The system also supports repeated judging with shuffled answer orderings. This feature helps measure whether the judge is stable or whether the ordering of candidates influences the result. In a reasoning evaluation task, stability matters because the final winner should depend on the quality of the answer, not on where it appears in the list.

Tie-breaking is handled carefully because many scientific questions produce close scores between candidates. When the judge scores are similar, the system uses verifier support, reasoning completeness, and supported claim counts to decide the winner. A deterministic fallback is also included so that the system behaves consistently across repeated runs. This is important because even small amounts of randomness can change the final decision if tie-breaking is not handled carefully. The final result is therefore more reproducible and more defensible.

The project also includes an interactive web interface designed to make the full process easier to inspect visually. Users can enter a question, choose the generation mode, enable or disable verification, and select a review style. The interface then displays the generated answers, the judge outputs, the verifier analyses, and the score breakdowns in a readable format. Instead of hiding the evaluation process, the interface makes it visible. This was an important design choice because the project is about reasoning transparency, not just final output quality. The interface also makes the system more useful for demonstration and experimentation because users can observe how the different parts of the framework work together.

III. EVALUATION

The evaluation for this project was conducted qualitatively using feedback collected through a Google Form survey. Rather than relying on benchmark scores or automated accuracy metrics, the goal of the evaluation was to understand

how real users perceived the system’s usefulness, design quality, reasoning quality, and overall idea. This approach was appropriate because the project is centered on transparency and explanation quality, both of which are difficult to capture with a single numeric metric. The qualitative evaluation therefore provided a way to study how users reacted to the system as an interactive reasoning tool.

The survey was shared with five participants, consisting of three family members and two friends. Each participant interacted with the application and explored the answer generation, judging, and verification features before submitting feedback. The form included both rating-based questions and open-response questions so that participants could give both quantitative impressions and qualitative comments. They were asked to rate the design, the evaluation idea, the reasoning provided by the judge, and whether they would want to use the application again in the future.

The results were encouraging. For the design-related rating question, three participants gave the project a rating of five out of five, while the remaining two participants gave a rating of four out of five. A similar pattern appeared in the responses about the evaluation idea itself. Most participants liked the concept of comparing multiple answers rather than relying on a single model response. Several participants remarked that the system felt more dependable because it combined generation, judging, and verification rather than depending on one stage alone. This suggests that the main idea of the project was understandable and appealing to users even outside the development context.

One of the most appreciated features was the verifier component used alongside the judge. Participants liked the fact that the verifier could add an extra layer of scrutiny and potentially identify unsupported claims that the judge might miss. This was important because it gave users the impression that the system was trying to protect them from weak or misleading reasoning rather than simply selecting the most polished response. Users also responded positively to the scoring process used for answers. The detailed scoring breakdown made it easier to understand why one answer was selected over another and gave the evaluation process a sense of structure and fairness.

The reasoning provided by the judge was also well received. Participants said that the judge explanations made the system feel more transparent and easier to trust. Instead of just seeing a final winner, they could see the reasoning behind the selection and the strengths and weaknesses of the different answers. This feedback is especially relevant because the project aims to make reasoning evaluation interpretable, not opaque. The fact that users responded positively to the judge explanations suggests that the system achieved that goal reasonably well.

Another strong result from the survey was that all five participants said they would like to use the application in the future. That response indicates that the system’s overall idea and presentation were appealing to users. Even though the sample size was small, the feedback still provided meaningful insight into which parts of the system stood out most. In

particular, the combination of multi-model comparison, verification, scoring, and visible reasoning was consistently seen as valuable. This makes the qualitative evaluation an important part of the project because it shows that the framework was not only technically functional but also understandable and useful to users.

Overall, the qualitative evaluation suggests that the project successfully delivered a transparent and appealing reasoning comparison system. Users responded positively to the verifier, the judge scoring, and the overall idea of comparing multiple model answers. The results also suggest that the interface and evaluation structure made the system more trustworthy and easier to interpret. Future evaluations could expand this study by involving more participants and testing whether the same positive feedback appears across different user groups.

IV. CONCLUSION

This project demonstrates a structured framework for evaluating scientific reasoning generated by large language models. Rather than focusing only on final answers, Cognitive Court emphasizes explanation quality, causal reasoning, mechanistic detail, and factual support. The system shows that model evaluation can be made more transparent when answer generation, verification, and judging are separated into distinct stages.

One of the most important lessons learned during development was that longer answers are not automatically better answers. A response may be verbose but still fail to provide meaningful scientific depth. For that reason, the judge rubric had to focus on mechanism coverage, explanatory depth, and logical coherence rather than on answer length. This helped ensure that the system rewards scientific understanding instead of surface-level detail.

Another important lesson was the value of modular design. By dividing the project into separate parts for filtering, generation, verification, judging, and visualization, the system became easier to debug and extend. This structure also made it easier to experiment with different modes and evaluation styles. In addition, the JSON handling logic proved essential because real model outputs are not always perfectly structured. The project therefore reinforced the idea that robust language-model systems need careful infrastructure around the models themselves.

The qualitative evaluation showed that users responded positively to the project, especially to the verifier and the scoring process. That feedback suggests that the system achieved its goal of making scientific reasoning evaluation more transparent and more trustworthy. Overall, Cognitive Court provides a strong foundation for future work on scientific answer comparison, reasoning evaluation, and interpretable model assessment. Future improvements could include larger user studies, benchmark-based evaluation, and deeper analysis of how different models perform across different kinds of scientific reasoning tasks.